University of Camerino

School of Science and Technology

Master Degree in Computer Science (LM-18) - Parallel and Distributed Programming

Computational Astrophysics: GPU-Accelerated Stellar Structure Identification in the Solar Neighbourhood using GAIA Data

Students
Toma Elia (132577)
Ferrara Ludovico (132550)
Bonsignore Christian (132016)

Professors
Ahmed Umair
Polini Andrea

A.A. 2025/2026

Abstract

This study addresses the computational challenges of unsupervised stellar clustering within the GAIA DR3 dataset by implementing high-performance GPU algorithms, specifically focusing on the efficient translation of density-based methodologies into massive parallel architectures. We present a comparative analysis of parallel configurations, evaluating the performance trade-offs between naive global memory access ($O(N^2)$) and optimized shared memory tiling strategies, utilizing thread cooperation to minimize bandwidth latency. Our results quantify the impact of architectural optimizations on execution time and memory throughput, demonstrating the efficacy of CUDA-accelerated computing for large-scale astronomical data mining. We observe significant speedups in neighbor search operations, confirming that memory-aware kernel design is essential for processing the expanding volumes of astrometric data.

Contents

1 Introduction **13**
1.1 Paper structure 13

2 Related Work **15**
2.1 DBSCAN 15
2.2 Friends of Friends 15
2.3 Problem Definition 15
2.3.1 Experimental Setup 16

3 DBSCAN and FoF Parallelization Strategy **17**
3.1 Parallel Implementation of DBSCAN 17
3.1.1 Phase 1: Graph Composition 17
3.1.2 Phase 2: Cluster Identification 18
3.2 Parallel Implementation of Single Linkage 18
3.2.1 Phase 1: Spatial Grid Construction 18
3.2.2 Phase 2: Boruvka Iterations (GPU Execution) 19
3.2.3 Phase 3: Post-Processing and Cluster Extraction 20

4 Results and Performance Analysis **21**
4.1 DBSCAN Evaluation 22
4.2 FoF Evaluation 22
4.3 BLOCK_SIZE Values Comparison 23

5 Conclusions **25**

Listings

2.1 ADQL query for Gaia DR3 local stellar dataset. 16

List of Figures

2.1 3D distribution of 5,000 stars from Gaia DR3. 16

3.1 Dendrogram generated by SLINK on 500 stars 19

4.1 Runtime Comparison — BLOCK_SIZE 256 21
4.2 Runtime Analysis of DBSCAN variants — BLOCK_SIZE 256 22
4.3 Runtime Analysis of FoF variants — BLOCK_SIZE 256 23
4.4 Analysis - Block Size 24

List of Tables

4.1 Performance (Execution Time in ms) on GAIA DR3 subset — BLOCK_SIZE 256 . . 21

1. Introduction

The transition of astrophysics from a purely observational discipline to a data-intensive computational science has necessitated the adoption of high-performance computing (HPC) architectures and advanced algorithmic strategies. The exponential growth in data volume has rendered traditional CPU-based serial processing insufficient, driving the adoption of Graphics Processing Units (GPUs) to accelerate discovery rates through massive parallelization [1] and real-time processing pipelines [2]. In the context of large-scale astrometry, this shift is exemplified by the implementation of hybrid MPI + CUDA solvers designed to handle sparse linear systems with 10^{11} non-zero elements for global sphere reconstruction [3]. Concurrently, the field has seen a surge in machine learning applications, ranging from Convolutional Neural Networks (CNNs) for maximizing the potential of spectroscopic surveys [4] to generative data-driven frameworks for spectral modeling [5]. However, the most critical computational challenge remains the unsupervised classification of stellar structures in high-dimensional phase space. Recent computational efforts have heavily relied on density-based clustering algorithms, such as HDBSCAN and single-linkage variants, to identify open clusters, associations, and halo substructures. These methods have been extensively applied to census improvements and membership identification [6] [7] [8], the detection of kinematic substructures and coronae [9] [10], and the exploration of integrals of motion space [11] [12].
The European Space Agency's GAIA mission represents the most significant data generation event in the history of Galactic astronomy, tasked with creating a precise three-dimensional map of approximately one percent of the Milky Way's stellar population [13] [14]. Since its initial data release [15], the mission has provided an unprecedented volume of astrometric, photometric, and spectroscopic data, necessitating robust data processing consortia to handle calibration and cataloging [16]. The most recent iteration, Data Release 3 (DR3), significantly expands the available feature set, offering full 6-dimensional phase space information (coordinates and velocities) [17] [18], which is essential for determining period-luminosity relations [19] and analyzing time-series data for over 10 million variable sources [20]. The granularity of this dataset allows for the detection of minute astrometric wobbles, facilitating the identification of sub-stellar companions and binary systems [21] [22], as well as the tracking of solar system objects like asteroids [23]. Consequently, GAIA DR3 serves not merely as a catalog, but as a massive, high-dimensional dataset that benchmarks the efficiency of modern parallel clustering algorithms.

1.1 Paper structure

This paper is organized as follows. Chapter 2 surveys the main GPU and clustering approaches for GAIA-scale stellar structure identification. Chapter 3 presents the proposed CUDA-based pipeline and the key design choices used to accelerate neighbor-search operations. Chapter 4 reports the performance results across representative problem sizes. Finally, Chapter 5 summarizes the contributions.

2. Related Work

2.1 DBSCAN

The DBSCAN algorithm (Density-Based Spatial Clustering of Applications with Noise), introduced by Ester et al., identifies clusters as dense regions of points separated by sparser areas, using two key parameters: ϵ (distance threshold) and $MinPts$ (minimum neighbors). It categorizes points as core, border, or noise, enabling the detection of arbitrary shapes without a predetermined cluster count, unlike K-Means. However, its $O(N^2)$ complexity is prohibitive for large datasets [24]. To address scalability, MR-DBSCAN employs MapReduce for skewed data, merging partial clusters from overlapping partitions to handle load imbalance [25]. Similarly, RP-DBSCAN uses a randomized cell partitioning strategy with a two-level dictionary to ensure uniform workload distribution regardless of data density [26]. For extremely large arbitrary datasets, NG-DBSCAN offers an approximate solution using Locality Sensitive Hashing to prune the search space, achieving near-linear scalability [27]. Alternatively, G-DBSCAN leverages GPU parallelism by reformulating clustering as a graph traversal problem, using iterative Breadth-First Search on an adjacency matrix to significantly outperform CPU implementations [28].

2.2 Friends of Friends

The Friends-of-Friends (FoF) algorithm is a foundational technique in astrophysics, originally utilized to identify virialized galaxy groups and dark matter halos based on spatial density enhancements [29] [30] [31]. Mathematically, FoF is equivalent to extracting flat clusters by applying a fixed distance threshold to a Single-Linkage clustering dendrogram, an approach historically optimized in $O(N^2)$ time and deeply rooted in the Minimum Spanning Tree (MST) formulations pioneered a century ago [32]. To handle increasingly massive datasets, computer scientists have introduced fully dynamic parallel algorithms for forest maintenance [33] and fast dual KD-tree structures to minimize neighbor query overheads [34]. Furthermore, the shift toward highly parallel architectures has driven the development of state-of-the-art GPU solvers; these include adaptive phase-space grid methods [35], massively concurrent MST constructions leveraging spatial binning [36], and highly optimized k-NN graph primitives designed to efficiently map hierarchical agglomeration onto CUDA cores [37].

2.3 Problem Definition

This project evaluates GPU-accelerated density-based clustering algorithms (DBSCAN and FoF) to identify stellar clusters among 2,234,316 stars in the local Galactic neighbourhood. Relying solely on static 3D spatial density, the implementation avoids $O(N^2)$ memory bottlenecks by implementing efficient memory management techniques like uniform domain decomposition. The parallel architecture is designed to safely handle boundary conditions and maintain high multiprocessor occupancy. Ultimately, performance and scalability are assessed by comparing GPU execution times and speedups against a sequential CPU baseline across datasets of increasing size.

2.3.1 Experimental Setup

Experiments were conducted in Google Colab using an NVIDIA Tesla T4 GPU to accelerate the CUDA kernels. The environment ran on a 64-bit Linux system with the GNU C++ compiler and the standard CUDA Toolkit, applying the -O3 optimisation flag for peak performance.
The dataset comprises 2,234,316 stars from the Gaia DR3 local solar neighbourhood, preprocessed into a 3D heliocentric Cartesian system in parsecs (Figure 2.1). To evaluate algorithm scalability, discrete subsets ranging from 1,000 to 200,000 stars were generated based on their Euclidean distance from the Sun.

```
SELECT TOP 200000000
      source_id,
      ra, dec, parallax
FROM gaiadr3.gaia_source
WHERE parallax > 5
   AND parallax_over_error > 10
ORDER BY parallax DESC
```

Listing 2.1: ADQL query for Gaia DR3 local stellar dataset.

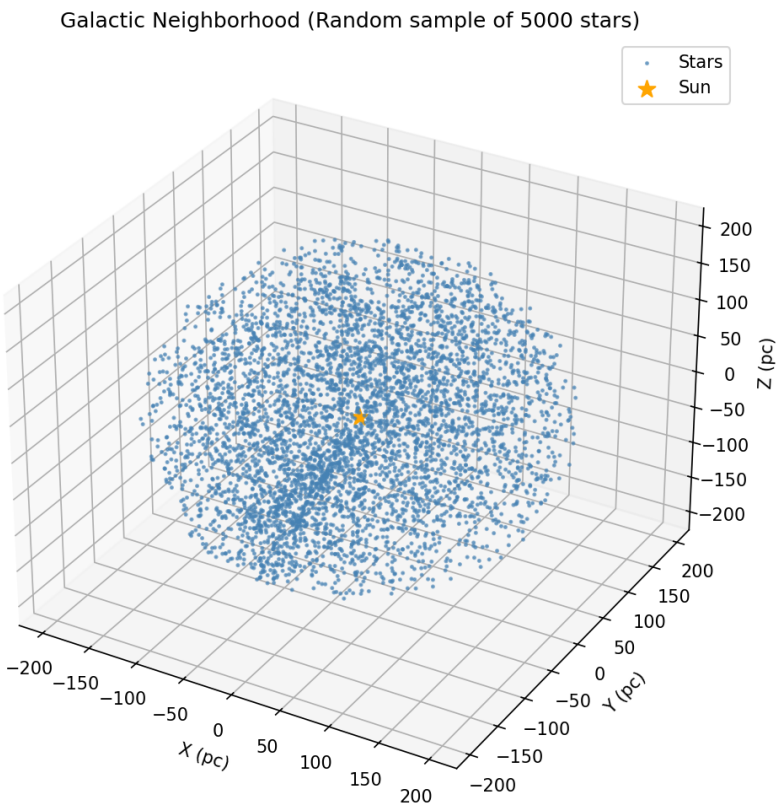

Figure 2.1: 3D distribution of 5,000 stars from Gaia DR3.

3. DBSCAN and FoF Parallelization Strategy

This chapter presents a high-performance, parallelized implementation of the DBSCAN and Friends-of-Friends algorithms (the second being basically the single linkage alorithm applied in astronomy [38]). Both of them are optimized to identify star clusters within large-scale 3D astronomical datasets like GAIA DR3.

3.1 Parallel Implementation of DBSCAN

Our approach leverages the G-DBSCAN methodology proposed by Andrade et al..[28]. By abstracting the data into a geometric graph structure, we decouple the workload into two distinct GPU-accelerated phases: a graph construction phase that maps celestial objects to a memory-efficient Compressed Sparse Row (CSR) adjacency model, and a cluster identification phase utilizing a parallel Breadth-First Search (BFS) [39]. This implementation exploits the massive parallelism to accelerate both the neighbor search (generating the graph edges) and the connected-component traversal, thereby enabling efficient analysis of high-density stellar regions that would be computationally prohibitive for traditional CPU-based solvers.

3.1.1 Phase 1: Graph Composition

The dataset is represented as a graph $G(V, E)$, where each point is a vertex and an edge exists between two points whose Euclidean distance is at most ε. This graph is stored in Compressed Sparse Row (CSR) format using two arrays: `row_ptr` (one entry per vertex, storing the offset into the neighbour list) and `col_indices` (the flat list of all neighbour indices). This compact representation minimises memory usage on the GPU, whose global memory is a critical constraint.

Step 1: Vertices Degree Calculation

One GPU thread is assigned to each point. Each thread counts how many other points fall within distance ε — this count is the vertex degree, and is written to a temporary `degrees` array. To reduce costly global-memory traffic, the kernel uses *shared-memory tiling*: each block cooperatively loads a tile of `BLOCK_SIZE` points into shared memory, so all threads in the block can reuse those reads when comparing against their own point. Since each thread operates independently on its own vertex there are no inter-thread dependencies, making this an embarrassingly parallel step. Its effective complexity is reduced from $O(V^2)$(sequential) to $O(V)$ on the GPU.

Step 2: Calculation of the Adjacency Lists Indices

The starting offset in `col_indices` for each vertex's neighbour list is determined by an *exclusive prefix sum* over the `degrees` array; the result is stored in `row_ptr`. This step has an inherent sequential data dependency (each entry depends on all prior ones), but is efficiently parallelised using Thrust's `thrust::exclusive_scan`, which implements an optimised parallel scan on the

GPU. The total number of edges is then recovered with minimal host–device transfer by reading only the last two scalar values of the scan output.

Step 3: Assembly of Adjacency Lists

With `row_ptr` fully computed, a second kernel (`construct_adjacency_list`) populates `col_indices`. Again one thread per vertex is used, and the same shared-memory tiling strategy is applied to reuse point data across threads. Each thread iterates over all tiles and, for each point within ε, writes that point's index into `col_indices` at the pre-computed offset `row_ptr[i]` plus a local counter. This mirrors the degree-counting kernel structurally, but writes neighbour indices instead of incrementing a counter, allowing both kernels to share the same tiling pattern.

3.1.2 Phase 2: Cluster Identification

Once the graph is built, a separate kernel classifies each point as *core* (degree $\geq$ `MinPts`, i.e. `row_ptr[i+1] - row_ptr[i] >= MinPts`) or non-core.
Cluster assignment then proceeds through repeated parallel BFS traversals, one per cluster.
BFS is executed level by level, following the approach of Harish & Narayanan (2007). Two boolean frontier buffers (`frontier_a` and `frontier_b`) are used in a ping-pong fashion. In each iteration, one GPU thread is assigned to every point. Threads whose point is on the current frontier expand to unvisited neighbours: they assign the current cluster ID to each unvisited neighbour and, crucially, only add that neighbour to the next frontier if it is itself a core point — border points receive the cluster label but do not propagate further. A shared boolean flag (`d_continue`) signals whether any new frontier nodes were added; the BFS loop on the CPU terminates when this flag remains false after a kernel launch.
The outer loop runs on the CPU: it iterates over all points, and whenever it finds an unvisited core point it seeds a new cluster and launches the BFS kernel sequence for that cluster. After each cluster's BFS completes, the cluster ID array is copied back to the host to update the seed-selection state. Both frontier buffers are cleared before seeding the next cluster. The overall complexity of the BFS phase is $O(V + E)$, giving a total algorithm complexity of $O(V^2)$ dominated by graph construction.

3.2 Parallel Implementation of Single Linkage

While density-based algorithms like DBSCAN extract a flat partition of the dataset based on a single spatial scale, hierarchical clustering offers a multi-scale perspective of the stellar distribution. Specifically, the Single Linkage agglomerative method constructs a hierarchy by iteratively merging the closest pairs of points or clusters.
To visually conceptualise the hierarchical merging process and the resulting nested structures within the stellar data, a subset of the dataset can be plotted as a tree diagram (Figure 3.1). However, as the dataset scales towards millions of stars, the $O(N^2)$ computational cost of sequential SLINK becomes entirely prohibitive. To achieve GPU acceleration, the clustering problem is mathematically reframed: computing the Single Linkage hierarchy is strictly equivalent to extracting the Euclidean Minimum Spanning Tree (EMST) of the dataset. Therefore, the parallel implementation abandons SLINK's approach in favour of a highly parallelised Boruvka-style algorithm, specifically adapted for 3D point clouds.
The parallel execution is divided into three distinct phases: spatial grid construction, iterative component merging on the GPU, and host-side tree filtering.

3.2.1 Phase 1: Spatial Grid Construction

To circumvent the computation of a full pairwise distance matrix, the host first constructs a uniform 3D spatial grid (domain decomposition) to bound the nearest-neighbour searches. The algorithm

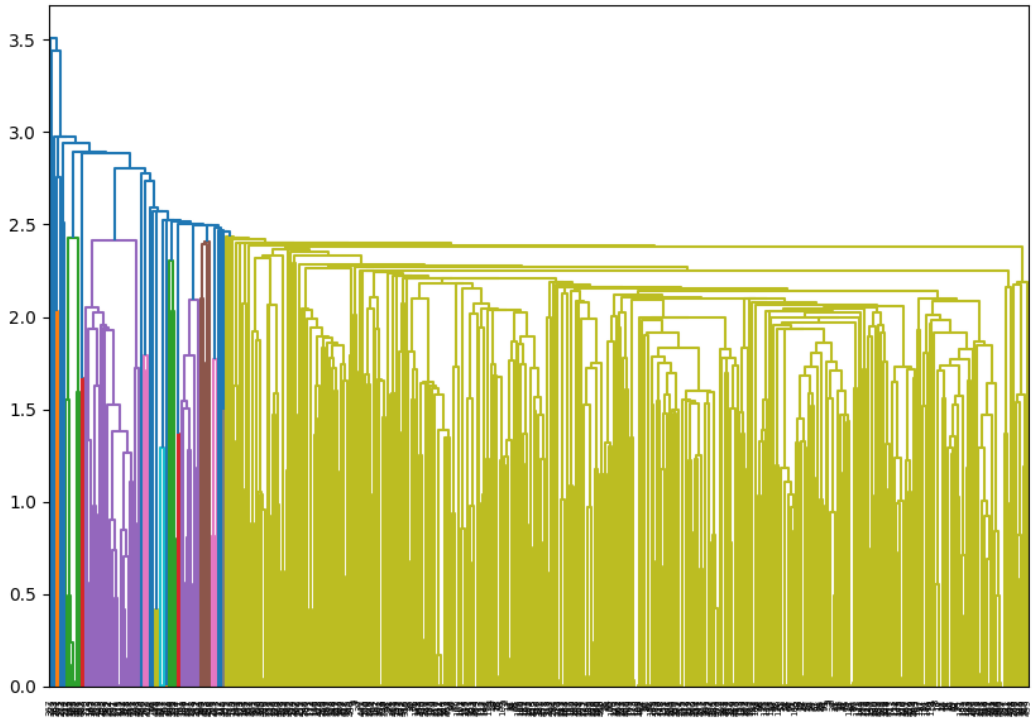

Figure 3.1: Dendrogram generated by SLINK on 500 stars

computes the axis-aligned bounding box of the point cloud and subdivides the space into a uniform grid with a resolution proportional to the cube root of the number of stars (N). This ensures that, on average, each cell contains approximately one point. A counting sort mechanism is then employed to group points by their flattened 3D cell ID. This preprocessing step provides the necessary spatial index arrays (‘cell_start‘ and ‘cell_count‘), allowing the GPU threads to efficiently localise searches without iterating over the entire dataset.

3.2.2 Phase 2: Boruvka Iterations (GPU Execution)

The core of the EMST computation occurs on the device through a series of Boruvka iterations. In each iteration, every connected component independently searches for its shortest outgoing edge to merge with a neighbouring component. This phase comprises a sequence of specific CUDA kernels:

Step 1: Finding the Closest Outgoing Edge

To exploit spatial locality, the search expands outward from the star’s host cell using concentric Chebyshev shells. The search terminates early if the minimum possible distance to the next shell exceeds the shortest distance already found, drastically reducing memory accesses.

Step 2: Component Minimum Edge Selection

Because a component generally contains multiple stars, the algorithm must identify the absolute shortest outgoing edge for the entire component. This is achieved through a custom 64-bit integer encoding: the high 32 bits store the floating-point distance, while the low 32 bits store the source index. A single ‘atomicMin‘ operation safely resolves race conditions, ensuring that the thread with the shortest valid distance successfully registers its edge as the component’s representative choice.

Step 3: Edge Collection and Merging

Once the optimal edges are identified, winning threads append them to a global MST edge array. To handle mutual nearest neighbours (where component A chooses B, and B chooses A), a deduplication check ensures that only the component with the smaller ID writes the edge. Subsequently, a merging kernel uses ‘atomicCAS‘ (Compare-And-Swap) on the component root nodes to safely link the merged components together, always pointing the larger component ID to the smaller one.

Step 4: Pointer Jumping

Following the merges, the component ID structure resembles a set of deep trees. A pointer jumping kernel flattens these trees, forcing each node to point directly to its grandparent (path compression). This kernel loops until a convergence flag confirms that the component IDs are fully flattened, concluding the iteration. The Boruvka loop repeats until all stars are unified into a single component or a predefined iteration limit is reached.

3.2.3 Phase 3: Post-Processing and Cluster Extraction

Due to the parallel nature of the Boruvka merges, the raw edge list generated on the device may occasionally contain redundant edges. To guarantee a strict tree structure, the raw edges are transferred back to the host and sorted by distance. A final Kruskal’s filtering pass is applied using a CPU-side Union-Find data structure with union-by-rank and path halving. This efficiently discards any duplicate or cycle-inducing edges, yielding the exact $N-1$ edges of the final Minimum Spanning Tree.

Finally, to extract discrete flat clusters comparable to those generated by DBSCAN, the algorithm simulates cutting the dendrogram at a specified spatial scale. By iterating through the filtered MST edges and removing any edge whose distance exceeds a predefined threshold (e.g., 2.0 parsecs), the single unified tree is split into a specific number of isolated sub-trees, each representing an independent stellar cluster.

4. Results and Performance Analysis

This chapter details the empirical evaluation of the implemented clustering algorithms, focusing on execution time and scalability across varying dataset sizes. To provide a rigorous comparative baseline, all timing measurements capture the total execution time in milliseconds to better illustrate performance variations at smaller scales. The computational benchmarks encompass the sequential CPU implementations alongside their massively parallel GPU counterparts, processing the Gaia DR3 subsets ranging from 1,000 to 2,234,316 stars.

#samples	NaiveDBSCAN	ParallelDBSCAN	SLINK	Boruvka-Elias
1000	1.902	2.974	31.558	297.572
10000	173.203	31.981	2755.69	29.63
50000	4752.65	209.844	81535.1	170.806
100000	23230.2	545.203	321103	575.037
200000	86233.7	1536.14	1297560	867.804
1000000	?	28611.4	?	47462.1
2234316	?	129175	?	270798

Table 4.1: Performance (Execution Time in ms) on GAIA DR3 subset — BLOCK_SIZE 256

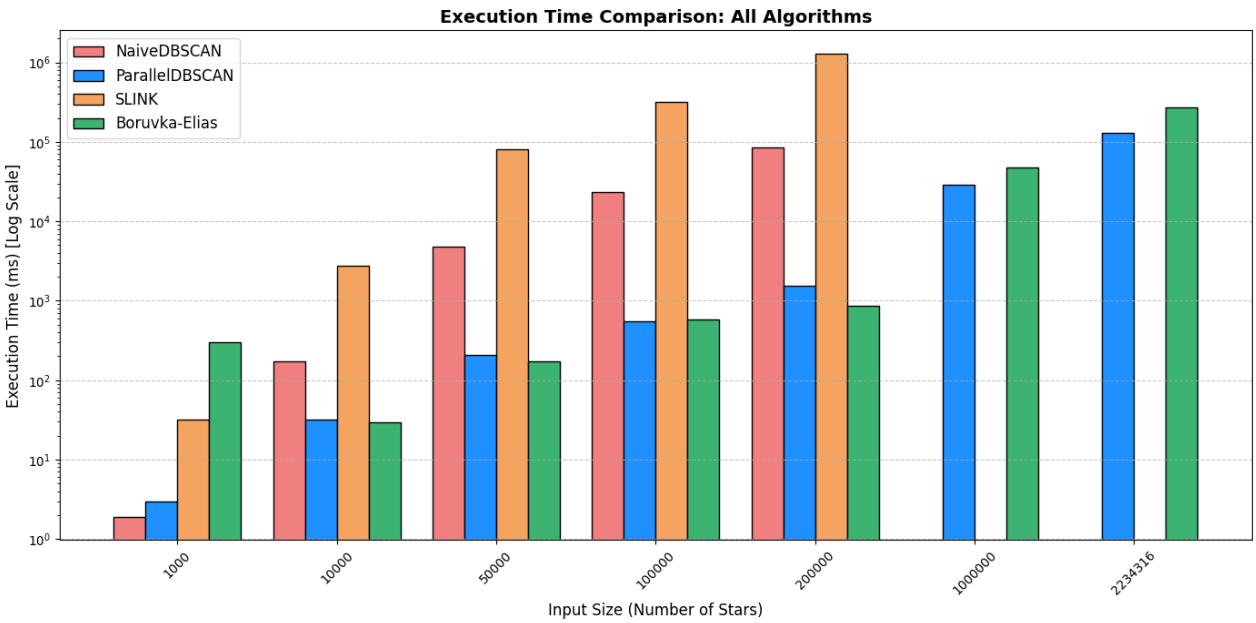

Figure 4.1: Runtime Comparison — BLOCK_SIZE 256

As illustrated in the preceding table and the accompanying grouped bar chart, a clear architectural divergence emerges as the dataset volume increases. The sequential algorithms exhibit severe scalability bottlenecks, becoming computationally prohibitive at the 200,000-star threshold. Conversely, the GPU-accelerated algorithms successfully absorb the geometric growth in computational complexity, managing to process the entire million-scale catalogue well within practical time limits.

4.1 DBSCAN Evaluation

The comparative analysis between the sequential Naive DBSCAN and the Parallel DBSCAN demonstrates the classical threshold behaviour associated with GPU acceleration, as visually confirmed by the line chart provided in this section. At the smallest experimental scale of 1,000 stars, the sequential CPU approach is marginally faster, executing in 1.90 milliseconds compared to the GPU's 2.97 milliseconds. This initial inversion is an expected hardware characteristic, reflecting the unavoidable overhead associated with CUDA context initialisation, memory allocation, and the host-to-device data transfer, which temporarily outweighs the benefits of parallelisation for trivial workloads.

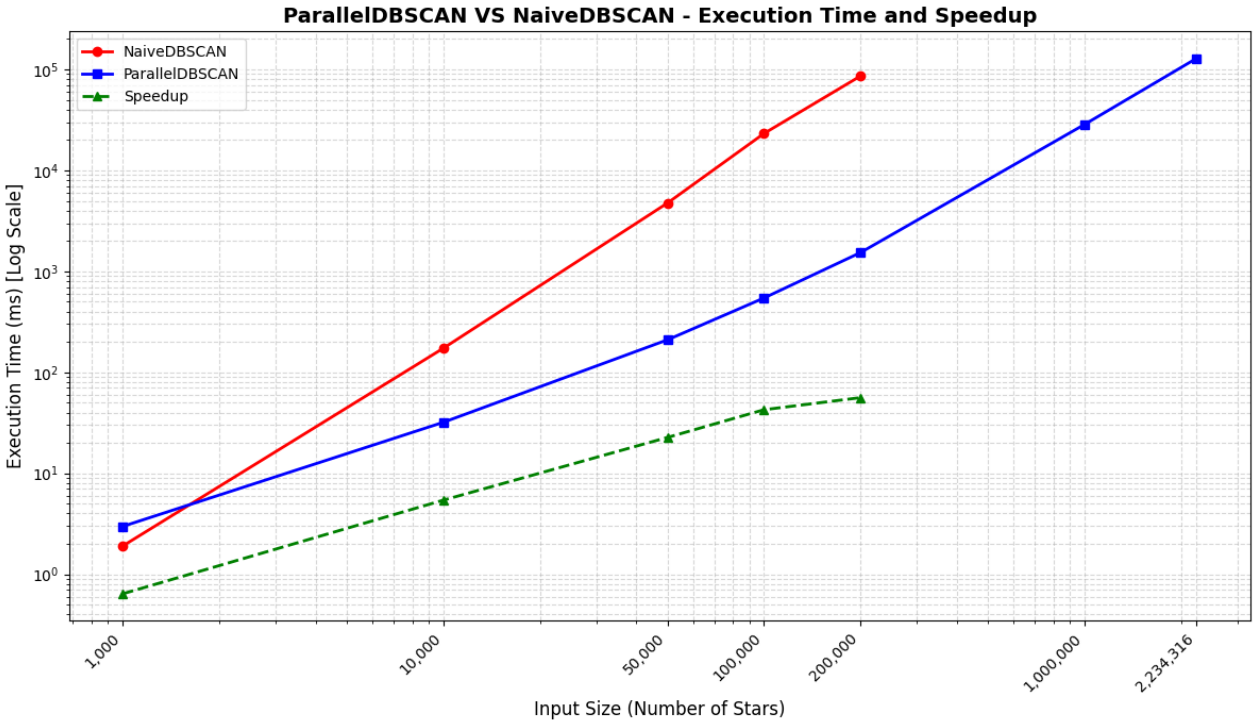

Figure 4.2: Runtime Analysis of DBSCAN variants — BLOCK_SIZE 256

However, the crossover point occurs rapidly. At a dataset size of 10,000 stars, the parallel implementation already outperforms the sequential baseline, finishing in approximately 31.98 milliseconds against the CPU's 173.20 milliseconds. From this point forward, the $O(N^2)$ algorithmic complexity of the unindexed sequential DBSCAN severely degrades its performance. By the time the input reaches 200,000 stars, the CPU requires roughly 86.2 seconds (86,233.7 milliseconds) to resolve the spatial density queries.

In stark contrast, the uniform domain decomposition and parallel neighbour search of the GPU implementation maintain an incredibly shallow growth curve. For the identical 200,000-star subset, the Parallel DBSCAN requires only 1.53 seconds (1,536.14 milliseconds). Furthermore, the parallel architecture proves robust enough to scale beyond the CPU's breaking point, seamlessly clustering the entire 2.2 million star catalogue in just under 130 seconds (129,175 milliseconds), validating the efficiency of the grid-based spatial indexing strategy.

4.2 FoF Evaluation

The evaluation of the Friends-of-Friends (FoF) algorithm juxtaposes the sequential SLINK method against the highly parallel Boruvka-Elias Minimum Spanning Tree approach. Observing the line chart corresponding to this section, the computational weight of building a full hierarchical structure becomes immediately apparent. The sequential SLINK algorithm is immensely demanding even at

low volumes, requiring 31.55 milliseconds for just 1,000 stars and degrading exponentially to a staggering 1,297.56 seconds (over 21 minutes) to process 200,000 stars.

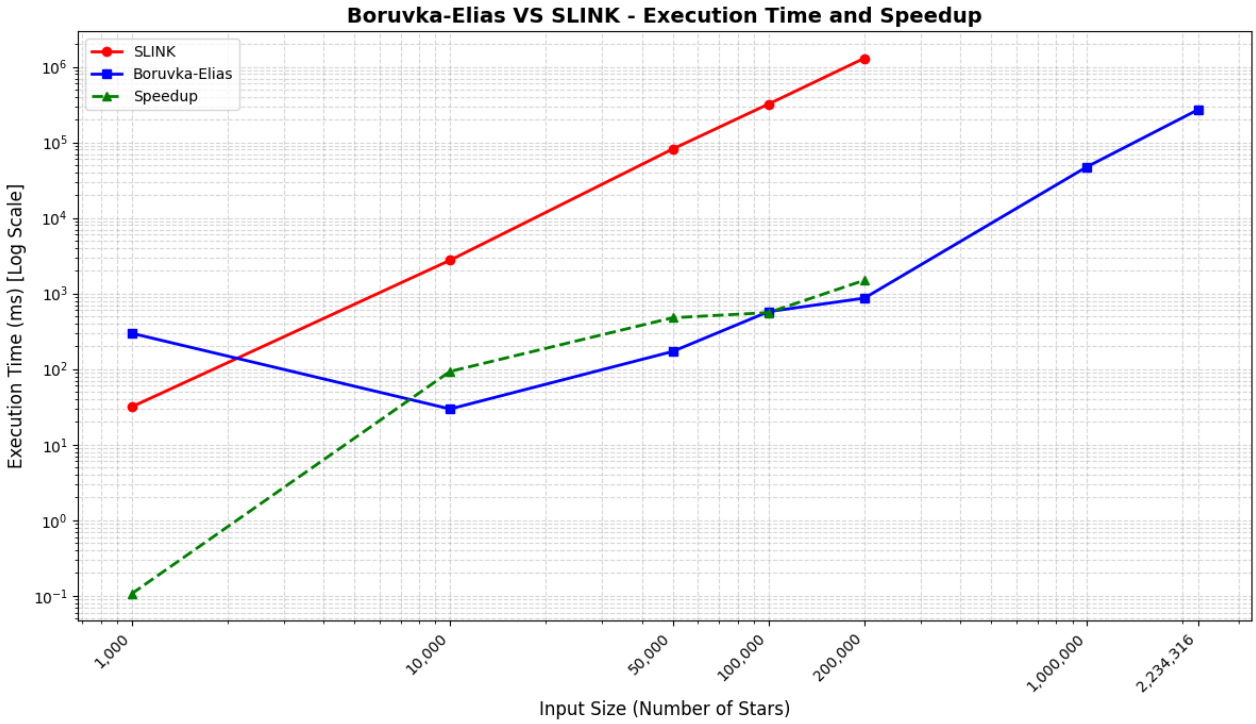

Figure 4.3: Runtime Analysis of FoF variants — BLOCK_SIZE 256

The parallel Boruvka-Elias implementation presents a fascinating anomaly at the 1,000-star mark, reporting an execution time of 297.57 milliseconds. This value is significantly higher than its subsequent run at 10,000 stars, which took a mere 29.62 milliseconds. This phenomenon is strictly attributable to the initial "warm-up" cost of the GPU; the first kernel launch incurs heavy context creation overhead, masking the actual algorithmic execution time on such a microscopic dataset. Once the GPU pipeline is primed, the true scalability of the parallel Euclidean Minimum Spanning Tree strategy is unveiled.

At the 200,000-star benchmark, the Boruvka-Elias algorithm completes the complex hierarchical merging process in just 867.80 milliseconds, rendering a task that took the CPU 21 minutes nearly instantaneous. Scaling up to the maximum capacity of 2,234,316 stars, the parallel algorithm constructs the entire forest and resolves the FoF clusters in approximately 270.79 seconds (roughly 4.5 minutes). While this is understandably slower than the flat partitioning of the Parallel DBSCAN—due to the heavier reliance on atomic operations, mutual nearest-neighbour deduplication, and pointer jumping required to resolve tree graphs—it represents a monumental leap in astrophysical data processing capabilities compared to traditional sequential clustering.

4.3 BLOCK_SIZE Values Comparison

Having established the immense scalability of the GPU-accelerated algorithms over their sequential counterparts, it is crucial to examine the hardware-specific tuning that dictates their peak efficiency. In the CUDA programming model, the execution configuration—specifically the BLOCK_SIZE (the number of threads per block)—profoundly impacts the occupancy of the Streaming Multiprocessors (SMs) and the ability to hide memory latency. To isolate and evaluate this behaviour, an experiment was conducted exclusively on the parallel implementations using the 1,000,000-star dataset, varying the block dimensions from 32 up to the architectural maximum of 1024 threads.

As depicted in the accompanying chart, the execution times for both algorithms exhibit a char-

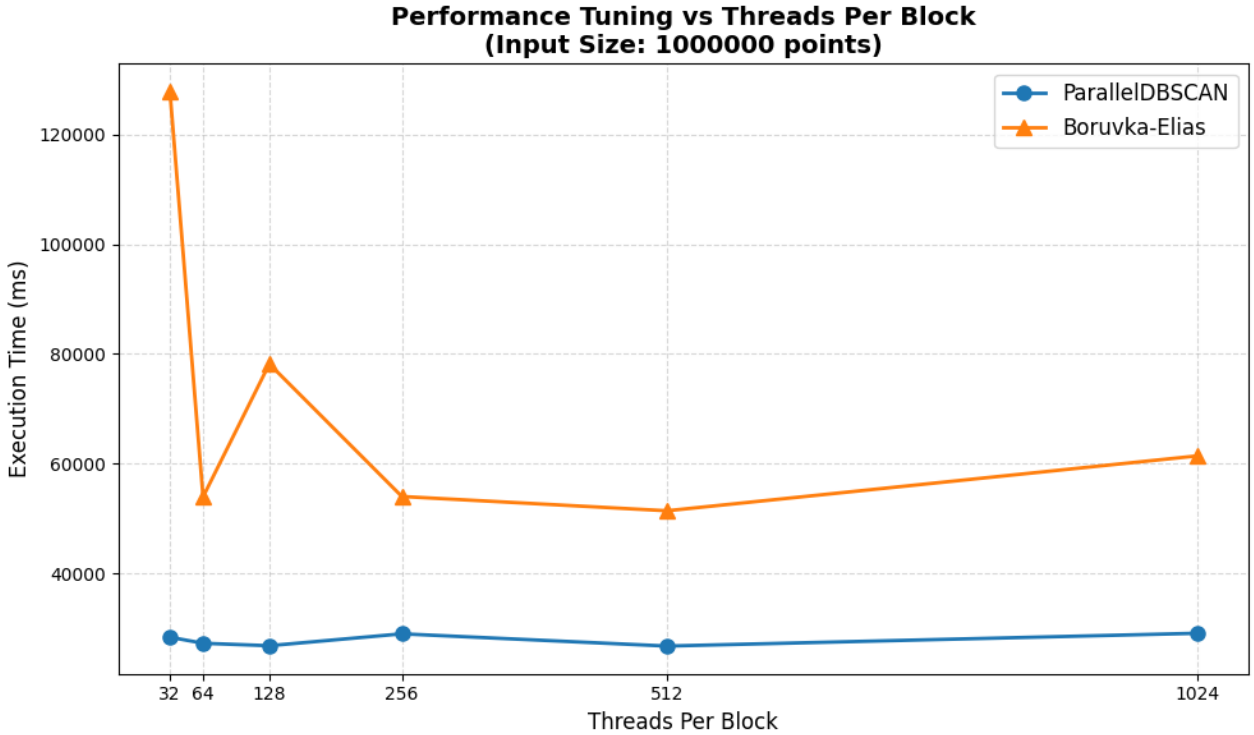

Figure 4.4: Analysis - Block Size

acteristic non-linear response to the block size adjustments. At the lower extremum (e.g., 32 or 64 threads per block), performance is suboptimal. This degradation occurs because allocating too few threads per block fails to provide the GPU scheduler with enough active warps to effectively hide memory access latency; when threads stall awaiting global memory fetches, the SM remains underutilised.

Conversely, as the block size increases, execution efficiency improves until it reaches a hardware-defined "sweet spot". For both the Parallel DBSCAN and the Boruvka-Elias FoF algorithms, this optimal configuration generally manifests around 256 or 512 threads per block. In this range, the GPU achieves maximum warp occupancy without exhausting the limited hardware resources—such as registers and shared memory—available per multiprocessor.

Pushing the BLOCK_SIZE to the maximum limit of 1024 threads often results in a slight performance penalty or plateau. This is particularly relevant for the Boruvka-Elias implementation. Because the Euclidean Minimum Spanning Tree logic relies heavily on atomic operations for component merging and requires substantial register allocation to maintain state variables during graph traversal, maximising the block size inevitably leads to register spilling or increased atomic contention. The Parallel DBSCAN, possessing a slightly more straightforward grid-lookup memory access pattern, demonstrates a more forgiving curve, though it similarly yields no further benefits beyond the optimal mid-range values.

Ultimately, this hardware tuning analysis empirically justifies the selection of 256 threads per block as the default configuration utilised throughout the broader performance evaluations in the preceding sections, ensuring an optimal balance between parallel concurrency and resource availability.

5. Conclusions

This study addressed the growing computational challenges associated with the unsupervised identification of stellar clusters by applying parallel algorithms to the massive astrometric dataset provided by the GAIA DR3 mission. The primary objective was to demonstrate that the transition towards massively parallel GPU architectures is not merely advantageous, but strictly necessary to overcome the scalability bottlenecks inherent in CPU-based approaches.

The comparative performance analysis confirmed the effectiveness of the adopted optimization strategies:

- **DBSCAN and memory optimization.** The reformulation of DBSCAN with the graph structure (G-DBSCAN) and the implementation of shared-memory tiling techniques successfully mitigated bandwidth latency. While the sequential version (Naive DBSCAN) becomes computationally prohibitive at just 1,000,000 stars, the parallel implementation demonstrated good scalability, processing the entire catalog of over 2.2 million stars in just under 130 seconds.
- **Friends-of-Friends (FoF) and Single Linkage.** For hierarchical clustering, abandoning the classical sequential SLINK approach (limited by $O(N^2)$ complexity) in favor of a Boruvka–Elias parallel algorithm for constructing the Euclidean Minimum Spanning Tree (EMST) generated an optimal speed-up. Thanks to spatial domain decomposition via uniform grids and use of atomic operations, the system processed 2.2 million stars in approximately 4.5 minutes (270 seconds). In contrast, the CPU baseline required over 21 minutes for a subset ten times smaller.
- **Architectural overhead management.** The tests also highlighted the expected hardware behavior, where for microscopic datasets (e.g., 1,000 stars) the CPU is marginally faster due to "warm-up" times and CUDA context initialization. However, the crossover point is rapidly surpassed, confirming that this overhead is negligible in real-world workloads.

In conclusion, the results clearly quantify the impact of a memory-aware kernel design. The adoption of efficient data structures (such as CSR adjacency lists and spatial grids) and thread cooperation proved essential for processing the expanding volumes of astronomical data mining. This work provides a robust and scalable CUDA pipeline capable of supporting the discovery of new stellar clusters and kinematic associations in the era of astrophysical Big Data.

Bibliography

[1] Christopher J. Fluke. *Accelerating the Rate of Astronomical Discovery with GPU-Powered Clusters*. en. arXiv:1111.5081 [astro-ph]. Nov. 2011. DOI: 10.48550/arXiv.1111.5081. URL: http://arxiv.org/abs/1111.5081.

[2] Alessio Magro. *A Real-Time, GPU-Based, Non-Imaging Back-End for Radio Telescopes*. en. arXiv:1401.8258 [astro-ph]. Jan. 2014. DOI: 10.48550/arXiv.1401.8258. URL: http://arxiv.org/abs/1401.8258.

[3] Valentina Cesare et al. "The MPI + CUDA Gaia AVU-GSR Parallel Solver Toward Next-generation Exascale Infrastructures". en. In: *Publications of the Astronomical Society of the Pacific* 135.1049 (July 2023). arXiv:2308.00778 [astro-ph], p. 074504. ISSN: 0004-6280, 1538-3873. DOI: 10.1088/1538-3873/acdf1e. URL: http://arxiv.org/abs/2308.00778.

[4] G. Guiglion. *Realising the potential of large spectroscopic surveys with machine-learning*. en. arXiv:2503.08196 [astro-ph]. Mar. 2025. DOI: 10.48550/arXiv.2503.08196. URL: http://arxiv.org/abs/2503.08196.

[5] Isabel Angelo et al. "A Data-driven Spectral Model of Main-sequence Stars in Gaia DR3". en. In: *The Astrophysical Journal* 974.1 (Oct. 2024), p. 43. ISSN: 0004-637X, 1538-4357. DOI: 10.3847/1538-4357/ad67db. URL: https://iopscience.iop.org/article/10.3847/1538-4357/ad67db.

[6] Emily L. Hunt and Sabine Reffert. "Improving the open cluster census. II. An all-sky cluster catalogue with Gaia DR3". en. In: *Astronomy & Astrophysics* 673 (May 2023). arXiv:2303.13424 [astro-ph], A114. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/202346285. URL: http://arxiv.org/abs/2303.13424.

[7] Penghui Liu et al. "Revisiting Open Clusters within 200 pc in the Solar Neighborhood with Gaia DR3". en. In: *The Astronomical Journal* 169.6 (June 2025), p. 326. ISSN: 0004-6256, 1538-3881. DOI: 10.3847/1538-3881/adceda. URL: https://iopscience.iop.org/article/10.3847/1538-3881/adceda.

[8] Jeison Alfonso, Alejandro García-Varela, and Katherine Vieira. "Exploring Galactic open clusters with Gaia I. An examination in the first kiloparsec". en. In: *Astronomy & Astrophysics* 689 (Sept. 2024). arXiv:2407.09407 [astro-ph], A18. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/202450901. URL: http://arxiv.org/abs/2407.09407.

[9] Xiaowei Ou, Lina Necib, and Anna Frebel. "Robust Clustering of the Local Milky Way Stellar Kinematic Substructures with Gaia eDR3". en. In: *Monthly Notices of the Royal Astronomical Society* 521.2 (Mar. 2023). arXiv:2208.01056 [astro-ph], pp. 2623–2648. ISSN: 0035-8711, 1365-2966. DOI: 10.1093/mnras/stad706. URL: http://arxiv.org/abs/2208.01056.

[10] Leslie Moranta et al. "New Coronae and Stellar Associations Revealed by a Clustering Analysis of the Solar Neighborhood". en. In: *The Astrophysical Journal* 939.2 (Nov. 2022), p. 94. ISSN: 0004-637X, 1538-4357. DOI: 10.3847/1538-4357/ac8c25. URL: https://iopscience.iop.org/article/10.3847/1538-4357/ac8c25.

[11] Haoyang Liu et al. *Exploration of Halo Substructures in IoM Space with Gaia DR3*. en. arXiv:2410.06646 [astro-ph]. Oct. 2024. DOI: 10.48550/arXiv.2410.06646. URL: http://arxiv.org/abs/2410.06646.

[12] Emma Dodd et al. "*Gaia* DR3 view of dynamical substructure in the stellar halo near the Sun". en. In: *Astronomy & Astrophysics* 670 (Feb. 2023), p. L2. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/202244546. URL: https://www.aanda.org/10.1051/0004-6361/202244546.

[13] Priya Hasan. *GAIA: The 3D Milky Way Mapper*. en. arXiv:2106.05125 [physics]. June 2021. DOI: 10.48550/arXiv.2106.05125. URL: http://arxiv.org/abs/2106.05125.

[14] Gaia Collaboration. "The Gaia mission". en. In: *Astronomy & Astrophysics* 595 (Nov. 2016). arXiv:1609.04153 [astro-ph], A1. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/201629272. URL: http://arxiv.org/abs/1609.04153.

[15] Gaia Collaboration et al. "*Gaia* Data Release 1: Summary of the astrometric, photometric, and survey properties". en. In: *Astronomy & Astrophysics* 595 (Nov. 2016), A2. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/201629512. URL: http://www.aanda.org/10.1051/0004-6361/201629512.

[16] L. Magrini et al. "The *Gaia* -ESO survey: Mapping the shape and evolution of the radial abundance gradients with open clusters". en. In: *Astronomy & Astrophysics* 669 (Jan. 2023), A119. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/202244957. URL: https://www.aanda.org/10.1051/0004-6361/202244957.

[17] Gaia Collaboration et al. "Gaia Data Release 3: Summary of the content and survey properties". en. In: *Astronomy & Astrophysics* 674 (June 2023). arXiv:2208.00211 [astro-ph], A1. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/202243940. URL: http://arxiv.org/abs/2208.00211.

[18] Tommaso Marchetti, Fraser A. Evans, and Elena Maria Rossi. "Gaia DR3 in 6D: The search for fast hypervelocity stars and constraints on the Galactic Centre environment". en. In: *Monthly Notices of the Royal Astronomical Society* 515.1 (July 2022). arXiv:2206.06962 [astro-ph], pp. 767–774. ISSN: 0035-8711, 1365-2966. DOI: 10.1093/mnras/stac1777. URL: http://arxiv.org/abs/2206.06962.

[19] Gisella Clementini. "Impact of the *Gaia* ESA mission on the primary Period–Luminosity Calibrators in the Milky Way: Cepheids and RR Lyrae". en. In: *Proceedings of the International Astronomical Union* 18.S376 (Dec. 2022), pp. 115–127. ISSN: 1743-9213, 1743-9221. DOI: 10.1017/S1743921323003411. URL: https://www.cambridge.org/core/product/identifier/S1743921323003411/type/journal_article.

[20] Laurent Eyer. *The Variable Sources in the Gaia archive*. en. arXiv:2412.02744 [astro-ph]. Dec. 2024. DOI: 10.48550/arXiv.2412.02744. URL: http://arxiv.org/abs/2412.02744.

[21] K.-U. Michel and M. Mugrauer. "Search for (sub)stellar Companions of Exoplanet Hosts by Exploring the Second ESA-Gaia Data Release". en. In: *Frontiers in Astronomy and Space Sciences* 8 (Mar. 2021), p. 624907. ISSN: 2296-987X. DOI: 10.3389/fspas.2021.624907. URL: https://www.frontiersin.org/articles/10.3389/fspas.2021.624907/full.

[22] M Mugrauer. "Search for stellar companions of exoplanet host stars by exploring the second ESA-Gaia data release". en. In: *Monthly Notices of the Royal Astronomical Society* 490.4 (Dec. 2019), pp. 5088–5102. ISSN: 0035-8711, 1365-2966. DOI: 10.1093/mnras/stz2673. URL: https://academic.oup.com/mnras/article/490/4/5088/5622591.

[23] B. Carry et al. "Potential asteroid discoveries by the ESA *Gaia* mission: Results from follow-up observations". en. In: *Astronomy & Astrophysics* 648 (Apr. 2021), A96. ISSN: 0004-6361, 1432-0746. DOI: 10.1051/0004-6361/202039579. URL: https://www.aanda.org/10.1051/0004-6361/202039579.

[24] Martin Ester, Hans-Peter Kriegel, and Xiaowei Xu. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise". en. In: ().

[25] Yaobin He et al. "MR-DBSCAN: a scalable MapReduce-based DBSCAN algorithm for heavily skewed data". en. In: *Frontiers of Computer Science* 8.1 (Feb. 2014), pp. 83–99. ISSN: 2095-2228, 2095-2236. DOI: 10.1007/s11704-013-3158-3. URL: http://link.springer.com/10.1007/s11704-013-3158-3.

[26] Hwanjun Song and Jae-Gil Lee. "RP-DBSCAN: A Superfast Parallel DBSCAN Algorithm Based on Random Partitioning". en. In: *Proceedings of the 2018 International Conference on Management of Data*. Houston TX USA: ACM, May 2018, pp. 1173–1187. ISBN: 978-1-4503-4703-7. DOI: 10.1145/3183713.3196887. URL: https://dl.acm.org/doi/10.1145/3183713.3196887.

[27] Alessandro Lulli et al. "NG-DBSCAN: scalable density-based clustering for arbitrary data". en. In: *Proceedings of the VLDB Endowment* 10.3 (Nov. 2016), pp. 157–168. ISSN: 2150-8097. DOI: 10.14778/3021924.3021932. URL: https://dl.acm.org/doi/10.14778/3021924.3021932.

[28] Guilherme Andrade et al. "G-DBSCAN: A GPU Accelerated Algorithm for Density-based Clustering". In: *Procedia Computer Science* 18 (2013), pp. 369–378. ISSN: 1877-0509. DOI: 10.1016/j.procs.2013.05.200.

[29] W. H. Press and M. Davis. "How to identify and weigh virialized clusters of galaxies in a complete redshift catalog". en. In: *The Astrophysical Journal* 259 (Aug. 1982), p. 449. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/160183. URL: http://adsabs.harvard.edu/doi/10.1086/160183.

[30] J. P. Huchra and M. J. Geller. "Groups of galaxies. I - Nearby groups". en. In: *The Astrophysical Journal* 257 (June 1982), p. 423. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/160000. URL: http://adsabs.harvard.edu/doi/10.1086/160000.

[31] M. Davis et al. "The evolution of large-scale structure in a universe dominated by cold dark matter". en. In: *The Astrophysical Journal* 292 (May 1985), p. 371. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/163168. URL: http://adsabs.harvard.edu/doi/10.1086/163168.

[32] Otakar Borůvka. "Borůvka, Otakar: Scholarly works". cs. In: ().

[33] Quinten De Man, Laxman Dhulipala, and Kishen N. Gowda. *Fully-Dynamic Parallel Algorithms for Single-Linkage Clustering*. en. arXiv:2506.18384 [cs]. June 2025. DOI: 10.48550/arXiv.2506.18384. URL: http://arxiv.org/abs/2506.18384.

[34] Y. Feng and C. Modi. "A fast algorithm for identifying friends-of-friends halos". en. In: *Astronomy and Computing* 20 (July 2017), pp. 44–51. ISSN: 22131337. DOI: 10.1016/j.ascom.2017.05.004. URL: https://linkinghub.elsevier.com/retrieve/pii/S221313371630066X.

[35] Daniel Wood. "Fast Galactic Structure Finding using Graphics processing units". en. In: ().

[36] Wen-Bao Qiao and Jean-Charles Créput. "GPU implementation of Borůvka's algorithm to Euclidean minimum spanning tree based on Elias method". en. In: *Applied Soft Computing* 76 (Mar. 2019), pp. 105–120. ISSN: 15684946. DOI: 10.1016/j.asoc.2018.10.046. URL: https://linkinghub.elsevier.com/retrieve/pii/S1568494618306148.

[37] Corey J. Nolet et al. *cuSLINK: Single-linkage Agglomerative Clustering on the GPU*. en. arXiv:2306.16354 [cs]. June 2023. DOI: 10.48550/arXiv.2306.16354. URL: http://arxiv.org/abs/2306.16354.

[38] Eric Feigelson. *Classification in Astronomy: Past and Present*. Ed. by Michael J. Way et al. 2012, pp. 3–10.

[39] Edward F. Moore. *The Shortest Path through a Maze*. Harvard University Press, 1959, pp. 285–292.